

JavaScript Array Method indexOf()

The `indexOf()` method is one of the most commonly used array methods in JavaScript. It is used to find the **index (position)** of a specific element in an array.

If the element is not found, it returns **1**.

Table of Contents

What is indexOf()? Why Use indexOf()? Syntax Parameters Return Value How indexOf() Works Basic Examples Multiple Examples Real Project Examples Best Practices Common Mistakes Interview Questions Summary Table

What is indexOf()?

Definition

The `indexOf()` method searches an array for a specified element and returns the **index of its first occurrence**.

If the element does not exist, it returns

1

It **does not modify** the original array.

Why Use indexOf()?

We use `indexOf()` when we need to:

Find an element's position Check if a value exists Remove an item using its index Update an element Search data inside an array

Syntax

```
array.indexOf(value);
```

```
array.indexOf(value, startIndex);
```

Parameters

Parameter	Description
value	The value to search for
startIndex	Optional. The index to start searching from

Return Value

Returns

Index of the first matching element **1** if the value is not found

How indexOf() Works

Array

```
["Apple", "Mango", "Orange"]
```

↓

```
indexOf("Mango")
```

↓

Result

```
1
```

Array

```
["Apple", "Mango", "Orange"]
```

↓

```
indexOf("Banana")
```

↓

Result

```
1
```

Example 1 (Basic)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.indexOf("Mango"));
```

Output

```
1
```

Explanation

"Mango" is located at index **1**.

Example 2 (Value Not Found)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.indexOf("Banana"));
```

Output

```
1
```

Example 3 (Numbers)

```
const numbers = [  
  10,  
  20,  
  30,  
  40  
];  
  
console.log(numbers.indexOf(30));
```

Output

```
2
```

Example 4 (Duplicate Values)

```
const numbers = [  
  10,  
  20,  
  20,  
  30  
];  
  
console.log(numbers.indexOf(20));
```

Output

1

Explanation

`indexOf()` always returns the **first matching index**.

Example 5 (Using startIndex)

```
const numbers = [  
  10,  
  20,  
  30,  
  20  
];  
  
console.log(numbers.indexOf(20,2));
```

Output

3

Explanation

Searching starts from index **2**, so the second **20** is found.

Example 6 (Empty Array)

```
const data = [];
```

```
console.log(data.indexOf(100));
```

Output

```
1
```

Example 7 (Case Sensitive)

```
const fruits = [  
  "Apple",  
  "Mango"  
];  
  
console.log(fruits.indexOf("apple"));
```

Output

```
1
```

Explanation

JavaScript is casesensitive.

Example 8 (Store Index)

```
const colors = [  
  "Red",
```

```
    "Green",  
  
    "Blue"  
];  
  
const index = colors.indexOf("Blue");  
  
console.log(index);
```

Output

```
2
```

Real Project Example 1

Remove a Product

```
const cart = [  
  
    "Laptop",  
  
    "Mouse",  
  
    "Keyboard"  
];  
  
const index = cart.indexOf("Mouse");  
  
cart.splice(index,1);  
  
console.log(cart);
```

Output

```
["Laptop", "Keyboard"]
```

Real Project Example 2

Check Student

```
const students = [  
  "Rahim",  
  "Karim",  
  "Rokon"  
];  
  
console.log(students.indexOf("Karim"));
```

Output

```
1
```

Real Project Example 3

Update Status

```
const status = [  
  "Pending",  
  "Approved",
```

```
    "Rejected"

];

const index = status.indexOf("Approved");

status[index] = "Completed";

console.log(status);
```

Output

```
[
  "Pending",
  "Completed",
  "Rejected"
]
```

Real Project Example 4

Search Category

```
const categories = [

  "HTML",

  "CSS",

  "JavaScript"

];

console.log(categories.indexOf("CSS"));
```

Output

1

Real Project Example 5

User Role

```
const roles = [

  "User",

  "Editor",

  "Admin"

];

console.log(roles.indexOf("Admin"));
```

Output

2

Best Practices

Use `indexOf()` when you need the position of an element.

Always check for `1` .

```
const index = array.indexOf(value);

if(index !== 1){

  console.log("Found");
```

```
}
```

Use meaningful variable names.

```
productIndex
```

```
studentIndex
```

```
userIndex
```

Common Mistakes

Mistake 1

Ignoring **1** .

Wrong

```
const index = fruits.indexOf("Banana");  
  
fruits[index] = "Apple";
```

Always check

```
if(index !== 1)
```

Mistake 2

Expecting the last occurrence.

```
const arr = [10,20,20];
```

```
console.log(arr.indexOf(20));
```

Output

```
1
```

Use `lastIndexOf()` for the last occurrence.

Mistake 3

Using `indexOf()` with objects.

```
const users = [  
  {  
    name: "John"  
  }  
];  
  
console.log(users.indexOf({  
  name: "John"  
}));
```

Output

```
1
```

Objects are compared by reference.

Mistake 4

Ignoring case sensitivity.

```
["Apple"].indexOf("apple");
```

Output

```
1
```

Interview Questions

What does `indexOf()` return?

The index of the first matching element.

What does it return if the element is not found?

```
1
```

Does `indexOf()` modify the original array?

No.

Can it search from a specific index?

Yes.

```
array.indexOf(value, startIndex);
```

Does `indexOf()` return the first or last occurrence?

The **first** occurrence.

Which method returns the last occurrence?

```
lastIndexOf()
```

Which method returns `true` or `false` instead of an index?

```
includes()
```

Summary Table

Feature	Description
Method	<code>indexOf()</code>
Purpose	Find the first index of a value
Parameters	value, startIndex (optional)
Returns	Index or <code>-1</code>
Changes Original Array	No
Common Uses	Search, Update, Remove, Validation

Quick Comparison

Method	Returns	Modifies Array
<code>includes()</code>	<code>true</code> / <code>false</code>	No
<code>indexOf()</code>	First Index / <code>-1</code>	No
<code>lastIndexOf()</code>	Last Index / <code>-1</code>	No

Final Notes

The `indexOf()` method is essential when you need the **position** of an element rather than just checking if it exists. It is widely used in:

Search functionality Updating array elements Removing items with `splice()` Data validation
Form processing Menu and navigation systems

Mastering `indexOf()` is an important step before learning more advanced array methods such as `find()` , `findIndex()` , and `filter()` .